Generative Modelling of Maritime AIS Trajectories

A Machine Learning Study across Twelve Model Iterations

Andrei Ştirbu
École Polytechnique
Supervised by Davide Buscaldi (LIX, École Polytechnique & Sorbonne Université)
Research Internship Report

May 2026

Abstract

This report studies generative modelling of maritime AIS trajectories through an iterative ladder of twelve transformer-based architectures, attacking two distinct prediction problems in sequence: (i) *next-step displacement prediction* (v1) – predicting the next position given recent history – and (ii) *conditional full-route generation* (v2 onward) – emitting a 128-waypoint route from only a start, end, and vessel type. The ladder spans next-step displacement and velocity models (v1), an encoder-decoder route generator with bearing features and scheduled sampling (v2–v5), an obstacle-conditioned branch (v6–v8), retrieval-augmented generation (v9), per-timestep retrieval with differentiable land-aware training and a windowed causal mask (v10), and two structural variants over discrete water graphs (v11–v12). The production system combines per-timestep retrieval over a 163-token memory with an A* post-processor on a 0.005° water-cell graph, achieving **Average Displacement Error (ade) of 6282 m** and **0 % land-crossing** at the 10 km threshold (n = 500, mean over 5 seeds). We provide post-mortems for six failed or parked variants that surface reusable ML design principles: per-step validation loss does not predict autoregressive rollout quality; constraint losses require constraint-violating training examples to fire; structural water-validity is decoupled from trajectory accuracy; and the candidate pool of any masked-softmax pointer must match the data's step-size distribution. A thirteenth iteration—a conditional denoising diffusion Transformer (TrajDiff)—is fully specified but not yet implemented.

Contents

1 Introduction and Context

1.1 Maritime trajectory modelling

Shipping carries over 80 % of world trade by volume, making maritime route intelligence a critical infrastructure. The Automatic Identification System (AIS) mandates that commercial vessels above 300 GT broadcast their position, speed, heading, and vessel category at intervals of a few seconds to several minutes. The US coastal coverage of the MarineCadastre dataset [7] logs approximately 170,000 vessels per day in the bounding box $[\text{lon } -125°, -60°] \times [\text{lat } 10°, 55°]$, providing a rich empirical record of actual traffic patterns.

Downstream consumers of route models include: route planners (planning fuel-efficient voyages), ETA estimators (forecasting port-arrival times), anomaly detectors (flagging routes that deviate from historical patterns), and digital-twin platforms such as **(author?)** [2], which require a synthetic route generator to stress-test event detectors. A survey of trajectory-prediction methods, from Kalman filters to recurrent networks to Transformer encoders, is given in **(author?)** [6].

1.2 Problem statement

Consider a sailor planning a route who must divert around a forming storm, a competing vessel, or a charted rock. The sailor knows — implicitly, from experience — that the diversion passes *behind* the obstacle, not in front of it; that it favours deeper water; that it rejoins the original course efficiently rather than zig-zagging. The technical problem we study is whether a model can learn this implicit maritime knowledge from historical AIS tracks alone.

The project pursued **two distinct prediction problems** in sequence, each occupying roughly half of the effort: (i) *next-step displacement prediction* (v1, Section 4.1) — given a history of recent positions for a single vessel, predict the immediate next position; and (ii) *conditional full-route generation* (v2 onward) — given only the start, end, and vessel type, emit a 128-waypoint route. The two problems share a feature representation but differ in their training signal (per-step displacement vs. rollout-quality of an entire sequence) and in their inference loop (one-shot vs. 128-step autoregressive). The lessons from (i) directly motivated the architectural choices in (ii).

Formally, for the route-generation problem, given

$$(\text{start} = (lon_0, lat_0), \quad \text{end} = (lon_T, lat_T), \quad v \in \{0, \ldots, 27\})$$

the system must emit a sequence of 128 geographic waypoints

$$\mathbf{x} = (x_1, \ldots, x_{128}), \qquad x_i = (lon_i, lat_i) \in \mathbb{R}^2,$$

with $x_1 \approx$ start, $x_{128} \approx$ end, and $\mathbf{x}$ resembling a plausible commercial shipping route. Plausibility has three levels in decreasing strictness: (1) *geographic admissibility* — the route lies on water; (2) *behavioural fidelity* — the route follows patterns typical for vessel type v; (3) *geometric quality* — the route is smooth and has correct total length.

1.3 Why this is hard

Three properties of the problem made this project longer than expected, and understanding them is essential to follow the design choices in Section 4.

Irregular constraint surface. Coastlines are GSHHS shoreline polygons [11] rasterised onto a grid; near-coast cells flip from "land" to "water" over ∼5 km, and many narrow inlets are technically water but commercially unreachable. Any model that attempts to enforce water-validity via a smooth loss is fighting against a discontinuous constraint.

Multimodal route distribution. A ship from Miami to Galveston can plausibly hug the Gulf coast, cut directly across the Gulf of Mexico, or avoid an offshore platform. AIS data records exactly one of these choices per voyage. A point-prediction model trained on this data will reproduce one mode—or worse, an average between two modes that is itself implausible—and the standard ADE metric measures distance to *one* ground-truth, not coverage of the mode set.

One-hundred-fold length variance. Straight-line distances in the dataset range from 20 km to 2000 km. Short routes are nearly straight and reward the great-circle baseline; long routes look like multi-leg voyages and reward retrieval-style memorisation. No single inductive bias serves both.

Noisy ground truth. Raw AIS data is heavily contaminated. The full prepare pipeline (vessel-type filtering, jump/gap filtering, turn segmentation, minimum speed/distance) discards roughly 97 % of raw pings before any modelling begins. Surviving tracks are still jiggly: vessels broadcast at irregular intervals, occasionally drop transmissions for minutes at a time, and individual tracks are not necessarily globally optimal routes (some loop, some take detours near ports, some are pieced together across multiple voyages of the same vessel). The data noise floor is therefore a hard upper bound on the precision any model can achieve. On top of this, the original 128-resampled dataset, built without coastal filtering, had ground-truth tracks that crossed land 36 % of the time at a 10 km SDF threshold and 87 % at a 0 km threshold — not a model bug, but a data issue arising from rasterisation noise and genuine coastal-river traffic. Cleaning the dataset (Section 4.7) is therefore best understood as an important prerequisite for fair architectural evaluation rather than as an architectural contribution per se.

1.4 Contribution overview

This report documents twelve model iterations across two prediction problems and proposes a thirteenth. The student's original ML contributions are:

- A systematic study of *next-step displacement prediction* (v1) — encompassing causal vs. bidirectional attention, displacement vs. velocity targets, lighthouse-distance and nearest-vessel auxiliary features — establishing that constant-velocity is a near-optimal baseline on post-turn-segmented open-sea tracks (Section 4.1). The exposure-bias and feature-redundancy lessons from v1 directly shaped the route-generation ladder that followed.

- An architectural ladder for *conditional route generation* (v2–v5): encoder-decoder seq2seq with learnable conditioning tokens, bearing features, scheduled sampling via a two-pass parallel formulation, and linear endpoint correction.

- *Retrieval-augmented decoding* (v9) and its per-timestep refinement (v10): a 163-token cross-attention memory built from the 5 nearest historical routes, combined with a windowed causal mask and a differentiable land loss applied through bilinear interpolation on a signed-distance raster.

- The WaterRouter post-processor (Section 4.6): A*- and BFS-based snapping on a fine-resolution water-cell graph.

- Post-mortems for six failed or parked variants (v6/v7/v8, v11, v12) identifying reusable ML failure modes: silent loss-clamping under data leaks; constraint losses with no constraint-violating training examples; candidate-pool / step-size mismatch in pointer architectures; and decoupling between structural guarantees and trajectory quality.

- The land signed-distance field infrastructure (Section 4.5.3) used for differentiable training loss and hard projection.

- The full data pipeline from NOAA AIS zips to 128-point route NPZ datasets (including the E1 land-filtering step, Section 4.7).
- The v13 TrajDiff engineering specification (Section 7).

Background material (Transformer architecture, DDPM, GSHHS, A* search) is cited throughout and clearly distinguished from original work.

1.5 Summary of all twelve approaches

Table 1 summarises every model iteration of the project; the remainder of this report explains each row in detail. A reader pressed for time can read only this table, the use-case ranking (Section 5.3), and the production pipeline command (Appendix A). Figure 1 shows the design lineage — which version's bottleneck motivated each next attempt. Status uses a fixed vocabulary: *production* (deployed), *validated* (works, superseded), *parked* (recoverable with future work), *abandoned* (structural failure, do not retrain), *planned* (specified, not yet built).

Table 1: All twelve trajectory model iterations plus the planned v13. Headline ADE for v1 is on the post-turn-segmented next-step setup (different from the v2+ route-generation metric); all v2+ numbers are 128-step autoregressive rollout ADE on $n = 500$ validation routes, seed=42. Asterisks (*) flag runs that stopped before the planned epoch count; training-duration notes are inline with each model in Section 4.

Ver.	One-line description	Status	Headline result
v1	Causal Transformer, next-step displacement regression on raw AIS pings	validated*	ADE matches CV baseline; cannot beat it
v2	Encoder-decoder seq2seq; predicts 128 waypoint deltas given (start, end, vessel-type); $\lambda_{\text{end}} = 50$	validated*	— (intermediate run, not evaluated end-to-end)
v3	v2 + scheduled sampling (warmup 20 ep)	validated*	— (intermediate)
v4	d=256, +bearing-to-end, 2 encoder layers	validated	— (val_delta only)
v5	v4 + scheduled sampling, two-pass parallel SS	validated*	ADE = 6753 m (ep 54/60)
v6	Encoder + K obstacle tokens, L_{obstacle} soft-repulsion	parked*	avoidance rate = 0 %; RNG-leak in augmenter
v7	v6 with the data leak fixed	parked*	structural failure (GT never crosses obstacles)
v8	Larger p_{obstacle}, richer augmenter	parked	not run; v7 analysis showed it would not help
v9	v4-encoder + K retrieved historical routes, mean-pool route encoder	validated*	ADE = 6207 m (ep 38/60, dirty data)
v10	v9 + per-step retrieval (K×t_retr tokens) + windowed causal mask + land-aware loss	**production***	ADE = 6282 m + 0 % land crossings (clean data, with WaterRouter)
v11	Halton-cone pointer over water-valid candidates	parked	ADE ≈ 30 km (step-size mismatch)
v12	Pointer over K-ring of fine water-cell graph	abandoned	ADE = 27,365 m; structural failure of CE-only loss
v13	Whole-sequence DDIM diffusion Transformer + CFG + length router	planned	TBD (Section 7 + docs/v13_proposal.md)

1.6 Evolution diagram

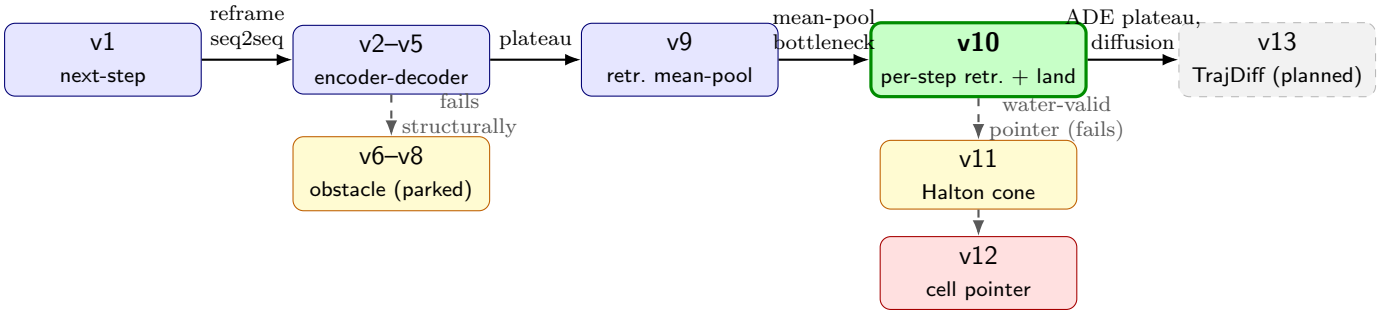


Figure 1: Design lineage of the twelve model iterations. Solid arrows denote successful continuations (the next version addresses a bottleneck identified in the previous one). Dashed arrows denote pivots into a parked or abandoned branch. Colour key: green = production, blue = validated/superseded, yellow = parked, red = abandoned, grey = planned. The main productive trajectory of the project is the horizontal spine v1 → v2–v5 → v9 → v10.

1.7 Notation and glossary

Project-specific terms are listed here for quick reference; all are defined in context where first used.

AIS Automatic Identification System. Mandatory ship-tracking radio signal broadcasting position, speed, heading, and vessel category at intervals of seconds to minutes.

MMSI Maritime Mobile Service Identity. Nine-digit numeric ID assigned to each AIS transponder; used as the train/val split key (by *root MMSI*) to prevent leakage between segments of the same physical vessel.

GSHHS Global Self-consistent Hierarchical High-resolution Shoreline. Vector shoreline dataset [11] from which the land-water boundary of this project is derived.

ade Average Displacement Error. Mean Euclidean distance (in metres) between predicted and ground-truth waypoints across all 128 time steps of all evaluated routes. The headline metric throughout.

fde Final Displacement Error. Euclidean distance between the predicted and ground-truth *final* waypoint. Driven to ≈ 0 by linear endpoint correction in all v2+ models.

Great-circle (GC) Geodesic arc baseline: 128 uniformly-spaced points on the shortest line between start and end. Geometric lower bound on route effort.

Retrieval-top-1 Zero-training baseline that returns, verbatim, the single nearest historical training route by 5-D KNN over (start, end, vessel-type).

Teacher forcing Training mode where the autoregressive decoder receives ground-truth positions as input at every step. Cheap in compute, but diverges from inference distribution.

Scheduled sampling Exposure-bias mitigation [1]: during training, each decoder step independently uses the model's own previous prediction with probability p_{ss} and the ground-truth otherwise.

Exposure bias The train/inference distribution mismatch caused by teacher forcing: the model has only ever seen ground-truth context at training time but must consume its own (noisier) predictions at inference, compounding errors over long rollouts.

Autoregressive (AR) rollout Inference procedure where the model generates one waypoint at a time, feeding each prediction back as input for the next step — in this project, 128 sequential decoder calls per generated route.

Causal mask Triangular attention mask in a Transformer that prevents position t from attending to positions $> t$, enabling next-token prediction.

Cross-attention Attention layer in an encoder-decoder Transformer where the decoder attends to the encoder's memory tokens; used here to consume the (start, end, vessel-type, retrieved-routes) conditioning.

Encoder-decoder Transformer Architecture pairing a bidirectional Transformer encoder (condition → memory) with a causal decoder (memory + past tokens → next token); the backbone of v2–v10.

Signed-distance field (SDF) Scalar field over $\mathbb{R}^2$ whose value at point p is the signed distance to the nearest land/water boundary. **Positive = land, negative = water** throughout this project.

Water-cell graph Boolean raster at 0.005° resolution (the fine grid) marking which cells satisfy $\text{sdf}_{\text{km}} \leq -0.5$, plus 4-connectivity edges between adjacent water cells. Domain of the WaterRouter A* search.

Linear endpoint correction After AR rollout, distributes the residual endpoint error $r = \text{pos}_{\text{gt}}^{T} - \text{pos}_{\text{pred}}^{T}$ linearly across all 128 waypoints (frac $i/(T-1)$). Drives FDE to ≈ 0 without retraining.

WaterRouter snap-only Post-processor that snaps each on-land predicted waypoint to its nearest connected fine-grid water cell via BFS (no inter-point routing). Production water-validity layer; see Section 4.6.

DDPM / DDIM Denoising Diffusion Probabilistic / Implicit Models [4, 9]: the diffusion framework underlying the planned v13 (Section 7).

Classifier-free guidance (CFG) Inference-time mixing of conditional and unconditional diffusion noise predictions [5] that strengthens conditioning at the cost of sample diversity. Planned for v13.

DiT Diffusion Transformer [8] — a Transformer block modulated by adaLN-Zero conditioning on the diffusion timestep t. The denoising backbone of v13.

2 Background

2.1 Transformer encoder and encoder-decoder

The Transformer [10] replaced recurrent networks as the standard sequence-modelling primitive. A *Transformer encoder* is a stack of identical layers; each layer applies multi-head self-attention followed by a position-wise feed-forward network (FFN). Self-attention computes, for each token i, a weighted sum of all tokens' values:

$$\text{Attn}(Q, K, V) = \text{softmax}\Big(\frac{QK^{\top}}{\sqrt{d_k}}\Big)\, V,$$

where Q, K, V are query, key, and value projections. A causal (triangular) mask restricts each token to attend only to past tokens, enabling autoregressive generation.

An *encoder-decoder* Transformer pairs an encoder that produces a context *memory* with a decoder that generates outputs token-by-token through cross-attention to that memory. This architecture underlies v2–v10 of this project.

2.2 Autoregressive generation and exposure bias

In autoregressive generation, the decoder produces position $t+1$ from all previously generated positions. During training with *teacher forcing*, the decoder receives ground-truth positions as input. At inference, it receives its own predictions, creating a distribution mismatch (exposure bias) that compounds errors over long rollouts. *Scheduled sampling* [1] mitigates this by probabilistically substituting self-predictions for ground-truth inputs during training.

2.3 Denoising diffusion models

Denoising Diffusion Probabilistic Models (DDPM) [4] define a *forward process* that gradually adds Gaussian noise to a data sample x_0:

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, I),$$

where $\bar{\alpha}_t$ follows a cosine schedule over $T = 1000$ steps. A neural network learns to predict ε (the "noise") given x_t and t. Denoising Diffusion Implicit Models (DDIM) [9] accelerate sampling to 20–50 steps by reusing predicted-noise estimates across timesteps. Classifier-free guidance (CFG) [5] mixes conditional and unconditional noise predictions at inference:

$$\hat{\varepsilon} = \varepsilon_{\text{uncond}} + w\,(\varepsilon_{\text{cond}} - \varepsilon_{\text{uncond}}), \quad w > 1,$$

steering the sample toward a conditioning signal. Diffusion Transformers (DiT) [8] replace U-Net backbones with Transformer blocks modulated by adaptive layer normalisation (adaLN-Zero).

2.4 Signed-distance fields

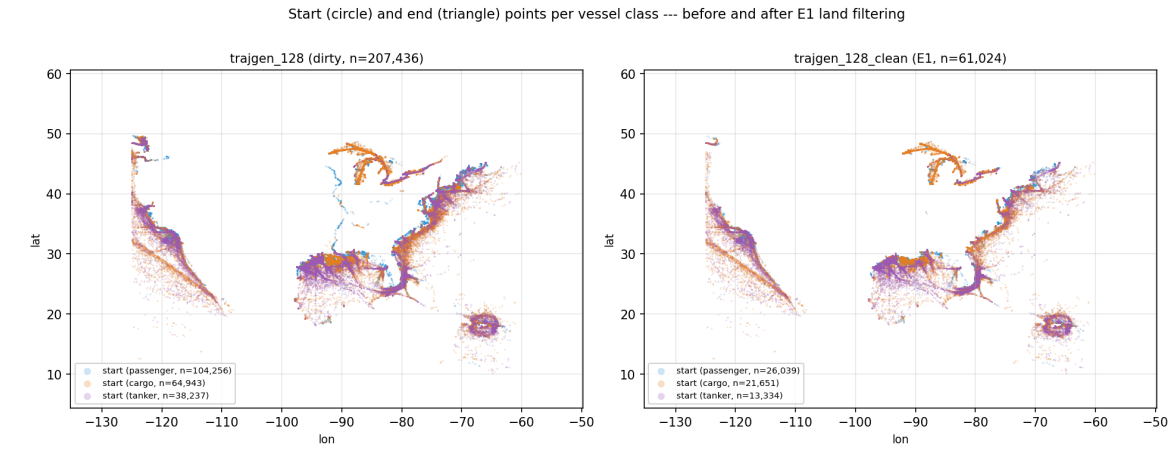
A signed-distance field (SDF) assigns to each point $p \in \mathbb{R}^2$ a scalar $\text{sdf}(p)$ whose absolute value is the shortest distance to a reference boundary ∂S, and whose sign indicates which side of the boundary p is on. In this project, **positive** values indicate land (inside the GSHHS polygon S) and **negative** values indicate water. The SDF is precomputed once from the GSHHS shoreline [11] at 0.05° resolution (Section 4.5.3).

3 Dataset and Experimental Setup

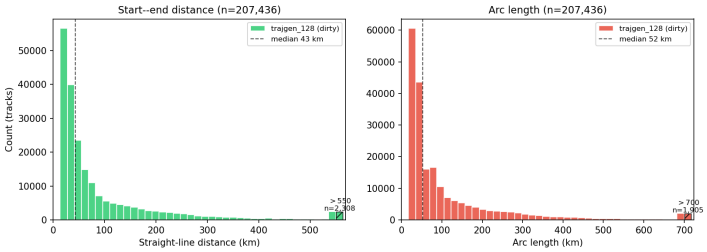
3.1 Data source and filtering

The dataset is twelve months of US-region NOAA AIS broadcasts from 2024, covering vessel types 60–89 (passenger, cargo, tanker, and similar commercial classes) within the bounding box lon $\in [-125°, -60°]$, lat $\in [10°, 55°]$. Raw pings are filtered by `prepare_data.py`: minimum track length 50 pings, maximum inter-ping jump 2 km, maximum gap 60 min, turn-segmentation at headings exceeding 150°, minimum average speed 1 km/h, minimum total distance 5 km. Approximately 97 % of raw pings are removed (80 % by vessel-type filter alone).

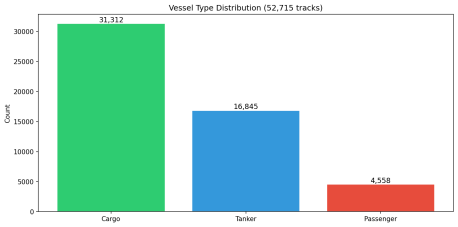
For the conditional route-generation models (v2 onward), tracks are resampled to exactly 128 points by uniform arc-length interpolation using `prepare_trajgen.py`, producing a dataset of 207,000 routes stored as `trajgen_128.npz`; each entry is a (128×2) array of normalised (lon, lat) values paired with metadata (start, end, vessel type). A cleaned subset `trajgen_128_clean.npz` is described in Section 4.7. The **next-step prediction models (v1, Section 4.1) operate directly on the raw irregularly-sampled tracks and do not resample**; their training corpus is the post-filter CSV `AIS_2024_gt80.csv` (tracks of >80 pings).



(a) Start (circle) and end (triangle) points of every track, coloured by vessel class, shown side-by-side for the dirty dataset (left, $n = 207\,436$) and the E1-cleaned subset (right, $n = 61\,024$). Cleaning removes inland-rivers and tight-coastal tracks without changing the overall coverage shape.



(b) Distributions of straight-line distance and arc length. X-axis clipped at the 99th percentile; the hatched bar on the right of each panel aggregates all tracks beyond the clip.



(c) Vessel-type mix in the 128-point dataset.

Figure 2: Dataset overview of `trajgen_128.npz`. 207k tracks spread across the US Atlantic, Gulf, and Pacific coasts; 45 % passenger, 24 % cargo, 14 % tanker; median straight-line distance 43 km with a long tail to 2000 km (Section 4 bucketed evaluation uses cutoffs 50 km and 500 km to separate these regimes).

3.2 Evaluation protocol

All trajectory-generation experiments are evaluated on a fixed set of $n = 500$ validation routes sampled with `seed=42`; results for the production system are additionally aggregated over the seed set $\{0, 1, 2, 42, 123\}$ in Section 5.3 and Appendix C to confirm the headline figure is not seed-cherry-picked. The train/validation split is by *root MMSI*: all AIS segments of the same physical vessel (identified by stripping batch prefix and segment suffix from the synthetic vessel ID) land entirely in train or entirely in validation, preventing data leakage.

Two primary metrics are reported throughout:

- **ade (Average Displacement Error)**: mean Euclidean distance in metres between predicted and ground-truth waypoints, averaged over all 128 steps and all 500 validation routes.
- **fde (Final Displacement Error)**: Euclidean distance in metres between predicted and ground-truth *final* waypoints. Linear endpoint correction (Section 4.2.3) drives FDE to near zero for all AR models.

Length-bucketed ADE further breaks down results into short (<50 km), medium (50–500 km), and long (>500 km) route categories.

Land-crossing metrics are: land%@d (fraction of predicted waypoints with sdf $> d$ km) and cross%@d (fraction of trajectories with at least one waypoint exceeding the threshold).

3.3 Hardware and software

Training runs were executed on heterogeneous hardware (CUDA GPU, Apple Silicon MPS, and CPU), handled by device auto-detection logic consistent across all model versions. All models are implemented in PyTorch. Inference latency for v10 + WaterRouter snap-only is approximately 50 ms per query on a single CPU core.

3.4 Baselines

Great-circle (GC): 128 uniformly-spaced points along the geodesic arc between start and end. ADE = 8226 m on `trajgen_128.npz`; serves as the lower-effort geometric baseline throughout.

Retrieval-top-1: returns the single most similar training route verbatim, selected by 5-D KNN over (start, end, vessel-type). ADE = 6470 m on `trajgen_128.npz` (seed=42 evaluation harness); a zero-training baseline showing the information value of historical traffic.

4 Technical Contributions

4.1 v1 – Next-step displacement prediction

4.1.1 Architecture

The first model treated trajectory modelling as a next-step regression problem: given the past $L = 120$ ping triples $(lon, lat, \Delta t)$, predict the displacement $(\Delta lon, \Delta lat)$ to the next ping. The implementation is a Transformer encoder with a causal (triangular) mask (`src/model.py`, class `AISTransformer`). Each timestep is encoded as a 5-dimensional vector:

$$x_i = [lon_i^{\text{norm}},\ lat_i^{\text{norm}},\ \log(\Delta t_i)^{\text{norm}},\ \Delta lon_i^{\text{norm}},\ \Delta lat_i^{\text{norm}}]$$

plus a learned 8-dimensional vessel-type embedding added at every step. Displacement is chosen as the regression target rather than absolute position because it has small, zero-centred variance independent of geographic location, keeping the per-coordinate target distribution stable across the dataset. Three independent scalers (position, log-Δt, displacement) are fitted on the training partition only.

Bidirectional attention is unsafe for next-step prediction. An early bidirectional variant (mean pooling of encoder outputs) attained much lower training MSE than the causal variant; closer inspection revealed the model was attending to *future* positions and degenerating into a copy operator, producing apparently-perfect training curves but useless rollouts. All subsequent v1 runs use causal masking, and the v2+ ladder uses teacher-forced training with autoregressive inference for the same reason (exposure bias is preferable to information leakage).

4.1.2 Iteration path

The v1 line is best understood as a sequence of targeted ablations, each motivated by a specific hypothesis about why per-step accuracy was not improving:

Stage 1 — causal vs. bidirectional. Initial training with bidirectional attention produced suspiciously low training MSE. The bidirectional model was reading future positions and copying them; switching to a causal mask immediately raised training MSE to a believable range while restoring meaningful rollout behaviour.

Stage 2 — displacement vs. velocity. A first round of training used absolute position as the regression target. Variance was dominated by geographic location (Atlantic vs. Gulf) rather

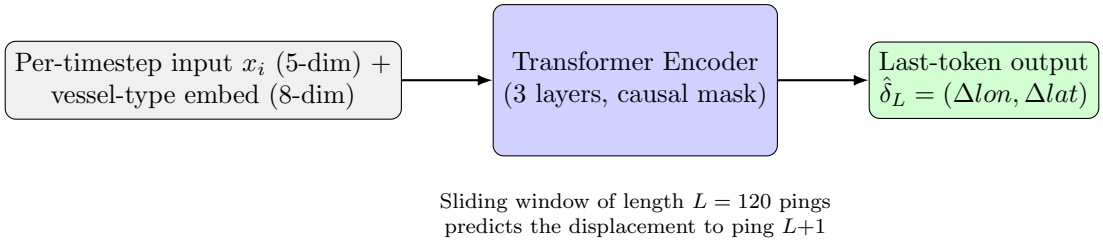


Figure 3: v1 next-step regression architecture. A causal Transformer encoder reads a 120-ping window and emits one displacement prediction from the last position's hidden state. The per-timestep input concatenates absolute position, $\log \Delta t$, the previous displacement, and a learned vessel-type embedding.

than by the actual prediction task, and gradients were correspondingly noisy. Switching to a *displacement* target $(\Delta lon, \Delta lat)$ centred the distribution at zero and stabilised optimisation. A further variant using *velocity* (displacement divided by Δt, so the target is ping-interval invariant) was trained in `train_vel.py`; it converged comparably but did not change the overall conclusion.

Stage 3 — vessel-type conditioning. A learned 8-dimensional embedding over the 28 AIS vessel codes is added at every timestep. Without it, the model defaults to passenger-vessel dynamics; with it, ablations confirmed cargo / tanker tracks were predicted with the expected lower turn rates.

Stage 4 — auxiliary features. Two auxiliary feature streams were prototyped: *lighthouse-distance* features (distances to the three nearest lighthouses, as a proxy for "how coastal is this ping"), and *nearest-vessel* features (distance and bearing to the nearest concurrent vessel, as a proxy for collision context). Both regressed performance: see Table 2, last row (lighthouse). The features were redundant with absolute position — the encoder could already infer "coastal" from (lon, lat) — and added noise without adding signal.

Stage 5 — multi-step rollout evaluation. A separate evaluation harness (`eval_multistep.py`) was added to predict horizons $H = 1, 2, 5, 10, 20$ autoregressively, accumulating errors. At every horizon, the model diverged *faster* than the constant-velocity baseline — a clear signature of exposure bias on a near-linear target. This was the result that prompted the move from per-step regression to full-sequence generation in v2.

4.1.3 Result

Table 2: v1 next-step model vs. constant-velocity (CV) baseline. ADE in metres; lower is better.

Run	CV ADE	Model ADE
12-month full dataset	50.7	50.8
Open-sea filtered	47.0	47.5
Open-sea + lighthouse	47.0	48.6

After all five iteration stages, the model matches but never beats the constant-velocity baseline (Table 2). Multi-step rollout diverged faster than CV at horizons 5, 10, and 20. Lighthouse features made results strictly worse; nearest-vessel features did not change them.

4.1.4 Lesson

Open-sea AIS tracks are nearly linear at 3-minute pings: the CV predictor is near-optimal on post-turn-segmented data. More broadly, **per-step accuracy in normalised MSE is not**

predictive of trajectory-level error. This finding recurs repeatedly through v2–v12 and is the single most important cross-cutting lesson of the project.

Training-duration note. Among the eight v1 runs, four reached their planned epoch count (`12mo_seq120`, `12mo_seq120_opensea`, `12mo_seq120_opensea_lh`, `4wk_seq80_e40`), and four were stopped early once the CV-tie conclusion was unambiguous: `12mo_seq120_bs384` (6/40, batch-size experiment), `12mo_seq120_lh` (15/40, lighthouse features regressed), `4wk_seq120_e30` (31/40, compute budget), and `12mo_seq80` (never run; queued config left for completeness). The headline number for v1 is the fully-trained `12mo_seq120_opensea` (40/40 epochs).

4.2 v2–v5 – Encoder-decoder trajectory generator

4.2.1 Reframing as seq2seq

The pivot from v1 to v2 reframed the task as a *sequence-to-sequence* generation problem: given (start, end, vessel_type), generate a full 128-point route. The dataset was rebuilt by `prepare_trajgen.py`, which resamples each track to exactly 128 arc-length-uniform waypoints and discards time-delta information (vessel speed is recovered implicitly through the vessel-type embedding).

4.2.2 Architecture (`src/model_gen.py`)

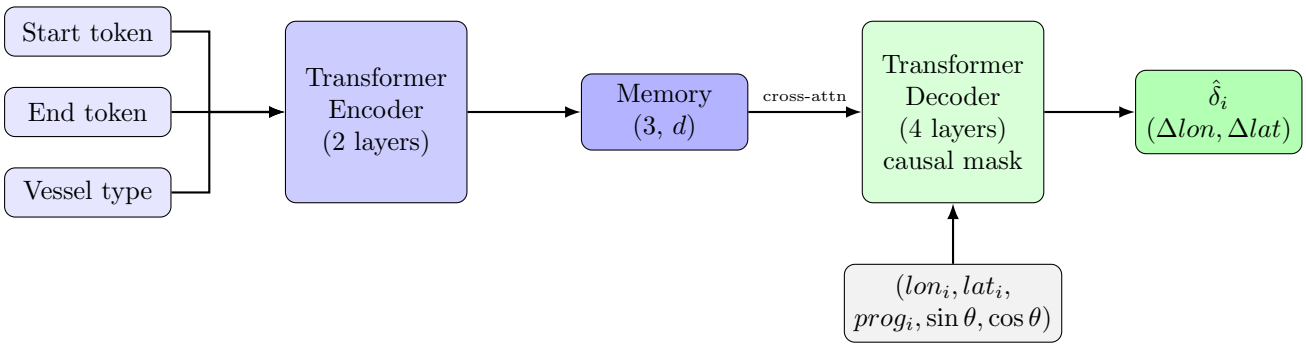


Figure 4: v4/v5 encoder-decoder architecture. The encoder produces 3 context tokens from start, end, and vessel-type conditioning. The decoder generates the 128-point route autoregressively via causal self-attention and cross-attention to the encoder memory. Input features include bearing-to-destination $(\sin\theta, \cos\theta)$ added in v4.

Encoder. Three conditioning tokens (start, end, vessel-type), each projected through a linear layer to d_{model}, plus learned token-type embeddings to distinguish their roles. Output: memory $\in \mathbb{R}^{3\times B\times d_{\text{model}}}$.

Decoder. Consumes a per-step input and generates 128 waypoints autoregressively. Each step's input (3-dim for MVP/v2/v3; 5-dim for v4/v5 with bearing) is projected to d_{model}, summed with sinusoidal positional encoding, and passed through four Transformer decoder layers (causal self-attention + cross-attention to encoder memory). A linear head emits predicted displacement $(\Delta lon, \Delta lat)$; the next position is recovered as:

$$\text{pos}_{i+1} = \text{pos}_i + \delta_{\text{scaler}}^{-1}(\hat{\delta}_i).$$

Loss.

$$\mathcal{L} = \mathcal{L}_\delta + \lambda_{\text{end}}\,\mathcal{L}_{\text{endpoint}} + \lambda_{\text{smooth}}\,\mathcal{L}_{\text{smooth}}, \tag{1}$$

where $\mathcal{L}_\delta$ is Huber loss on predicted vs. ground-truth deltas, $\mathcal{L}_{\text{endpoint}} = \|\text{pos}^T_{\text{pred}} - \text{pos}^T_{\text{gt}}\|^2$, and $\mathcal{L}_{\text{smooth}} = \frac{1}{T-2}\sum_i \|\text{accel}_i\|^2$ penalises second-order acceleration. MVP configuration: $\lambda_{\text{end}} = 10$, $\lambda_{\text{smooth}} = 1.0$.

4.2.3 Linear endpoint correction

After autoregressive rollout, the residual endpoint error $r = \text{pos}_{\text{gt}}^{T} - \text{pos}_{\text{pred}}^{T}$ is distributed linearly across all 128 waypoints:

$$\text{pos}_i^{\text{corrected}} = \text{pos}_i^{\text{raw}} + r \cdot \frac{i}{T-1}, \quad i = 1, \ldots, 128.$$

This drives FDE to near zero without retraining and is retained in every subsequent model.

Algorithm 1 Linear Endpoint Correction

Require: raw trajectory $\mathbf{x}^{\text{raw}}$, ground-truth endpoint $\text{pos}_{\text{gt}}^{T}$

$r \leftarrow \text{pos}_{\text{gt}}^{T} - x_{128}^{\text{raw}}$
for $i = 1$ **to** 128 **do**
  $x_i^{\text{corrected}} \leftarrow x_i^{\text{raw}} + r \cdot (i-1)/(128-1)$
end for

return $\mathbf{x}^{\text{corrected}}$

4.2.4 Scheduled sampling (v3, v5)

To mitigate exposure bias, v3 and v5 use *scheduled sampling* [1]. The naive step-by-step replacement was infeasible at $T = 128$, so the implementation uses a *two-pass parallel* scheme: Pass 1 runs the standard parallel teacher-forced decoder under `torch.no_grad()` to obtain one-step-ahead predictions; a mixed input trajectory is constructed by selecting ground-truth or self-prediction per step with probability p_{ss}; Pass 2 re-runs the decoder with gradients on the mixed input. Cost: $\approx 2\times$ standard teacher forcing vs. $\sim 128\times$ for the naive loop.

4.2.5 Architectural ladder results

Table 3: v2–v5 architectural ladder. val_delta is per-step loss in normalised delta space; ADE is evaluated by autoregressive rollout over 500 validation routes (seed=42). The architectural ladder produced no real-world ADE improvement.

Config	Changes vs. MVP	d_{model}	Enc. layers	val_delta	ADE (m)	
MVP	Baseline	128	0	0.086	6807	
v2	$\lambda_{\text{end}} = 50$	128	0	0.105	—†	† v2
v3	v2 + SS	128	0	0.105	—†	
v4	d=256, bearing feat., 2 enc. layers	256	2	0.072	6801	
v5	v4 + SS	256	2	0.074	6753	

and v3 used the earlier 3-dim decoder input (no bearing feature); the live `TrajectoryGenerator` class no longer accepts those checkpoints. Per-step val_delta is reported instead; the rollout-ADE conclusion is unchanged by v4's measurement (6801 m, statistically indistinguishable from MVP at 6807 m).

Figure 5 and Table 3 document the central negative finding: a 14 % improvement in val_delta (0.086 → 0.074) corresponds to a ∼1 % improvement in ADE (6807 → 6753 m) that vanishes inside evaluation noise. The entire architectural ladder produced no measurable real-world improvement.

Figure 6 shows the qualitative failure mode that the architectural ladder did not address: v5 routes whose *final waypoint* terminates inland, several tens of kilometres from any coastline.

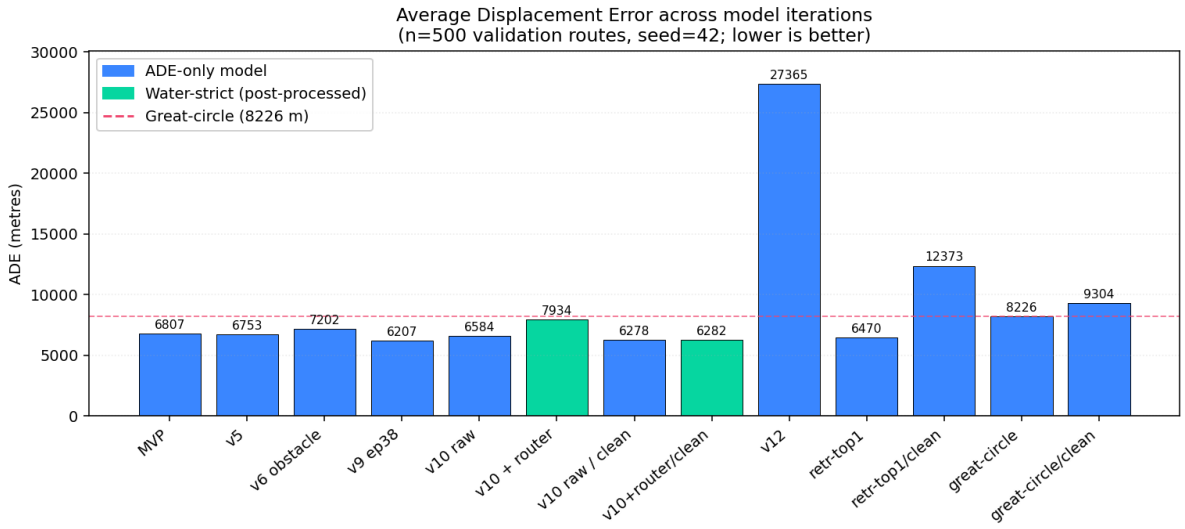


Figure 5: ADE across model iterations. Blue bars: ADE-only models on `trajgen_128.npz` (dirty data). Green bars: water-strict variants. Dashed red line: great-circle baseline. Note the logarithmic scale on the right portion of the x-axis for v12.

These cases are typical of medium-length val routes where the geometry of nearby historical traffic is the missing signal — and are what motivated the move to retrieval-augmented generation in v9.

Training-duration note. Of the five v2–v5 runs, only `trajgen_v4` reached its planned 60 epochs. `trajgen_mvp` stopped at 45/50 once the val_delta plateau was unmistakable; `trajgen_v2` and `trajgen_v3` stopped at 31/100 and 20/100 respectively because their plateau was visible even earlier; `trajgen_v5` stopped at 54/60 because the ladder's lesson (no real-world ADE movement) was already established and further training was deemed unlikely to alter the headline.

4.3 v6, v7, v8 – Obstacle-conditioned generation (parked)

4.3.1 Motivation and architecture

v6/v7 extended the v4/v5 encoder with $K \leq 3$ obstacle tokens, each encoding (centre_lon, centre_lat, radius_deg) projected to d_{model}. The training loss added a fourth term:

$$\mathcal{L}_{\text{obs}} = \frac{1}{T} \sum_{i} \max(0,\ r + m - \|x_i - c\|)^2 \,, \tag{2}$$

a one-sided quadratic penalty firing when predicted waypoint x_i penetrates obstacle circle (centre c, radius r, margin m). The intent was obstacle-avoidance: given exclusion zones, the decoder should route around them.

4.3.2 v6 — the data leak

`ObstacleTrajectoryGenDataset.__getitem__` seeded its RNG with `self.seed + idx`, assigning the *same* obstacle placement to every sample at every epoch. The model memorised (trajectory, obstacle) pairs; the ground-truth route never crossed those obstacles; $\mathcal{L}_{\text{obs}} \equiv 0$ throughout training. The training log showed `val_obstacle` $\approx 3 \times 10^{-5}$—not "avoidance working", but "the penalty had nothing to penalise". Evaluation returned an avoidance rate of 0.0 %.

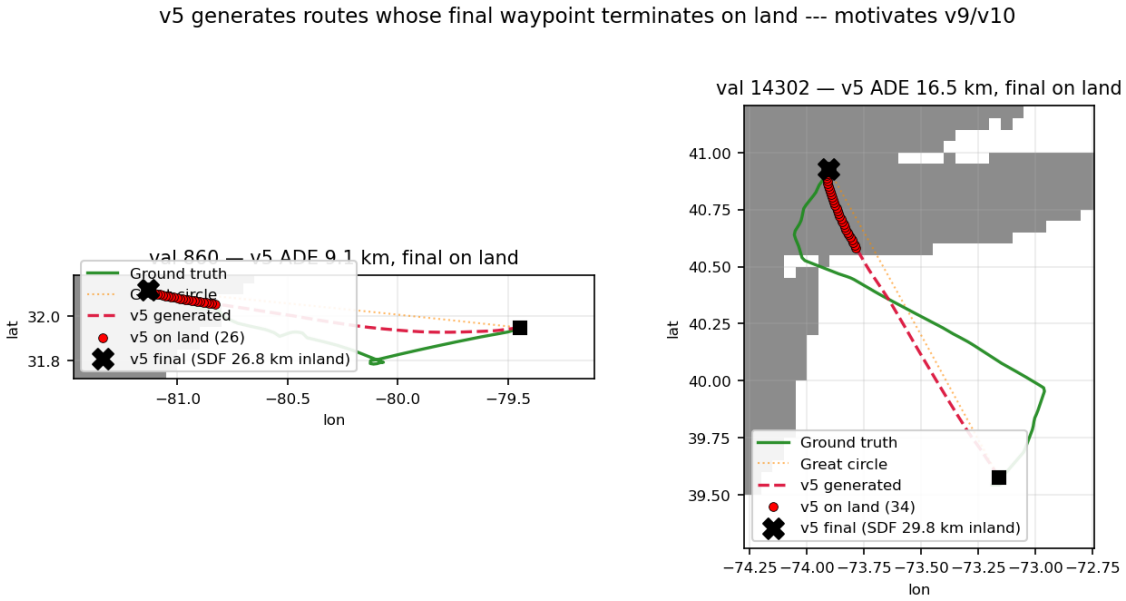


Figure 6: Two v5 (epoch 54) predictions whose final waypoint terminates on land (signed-distance to coast $>$ 25 km inland in both cases). Grey blocks are land masses (rasterised GSHHS coastline at 0.05°); white is water. Green solid is the ground-truth route, red dashed is the v5 prediction, dotted orange is the great-circle baseline, and the black `X` marks the v5 final waypoint. Red dots flag every v5 waypoint with sdf_km $>$ 0 (on land). The model has no explicit notion of where land is; nothing in the per-step displacement loss penalises a final-waypoint excursion into the continental interior. Land awareness is added in v10 (Section 4.5).

4.3.3 v7 — the structural failure

Fixing the data leak (`deterministic=False` in the dataset) revealed a deeper problem: the obstacle augmenter placed obstacles *laterally offset* from the ground-truth route, not on it. The GT route therefore never approached the obstacle; $\mathcal{L}_{\text{obs}} = 0$ on any decoder output close to GT. The decoder cross-attended to the obstacle tokens but they carried no predictive signal; spurious epoch-specific correlations drove val_loss from 0.72 (epoch 5) to 14.29 (epoch 17).

Structural diagnosis. A constraint loss can only produce a gradient on training examples where the constraint is potentially *violated*. Reviving obstacle-avoidance requires synthesising true detour trajectories (e.g., bent great-circle routes, RRT-planned detours) as ground-truth targets—a substantial dataset-engineering project not attempted in this work.

Training-duration note. `trajgen_v6_obstacle` ran 50/60 epochs before the RNG-leak was found; `trajgen_v7_obstacle` ran 22/60 after the leak fix, at which point the val-loss explosion (Section 4.3) made the structural problem unambiguous and further training was halted. v8 was a planned hyperparameter sweep (`p_obstacle`: 0.5 → 1.0, richer augmenter, scheduled sampling) that was never run after the v7 analysis showed augmenter-side fixes could not address the zero-gradient root cause.

4.4 v9 – Retrieval-augmented generation

4.4.1 Retrieval-top-1 gate

Before training v9, a zero-parameter experiment was run: for each validation query, return the nearest training route verbatim (retrieval-top-1), where nearness is defined by 5-D KNN over

(start_lon, start_lat, end_lon, end_lat, vessel_type). Result: ADE = 5489 m[1], a 19 % improvement over the best trained model (v5 at 6753 m). This zero-training baseline validated the retrieval substrate before any parameters were learned.

4.4.2 Architecture (`src/model_gen_retrieval.py`)

v9 extends the v4 encoder with $K = 5$ retrieved historical routes. A `MeanPoolRouteEncoder` compresses each retrieved (128×2) route to a single d_{model}-dimensional vector by mean-pooling the 128 position vectors and projecting. Encoder memory grows from 3 base tokens to $3+5 = 8$ tokens. The decoder cross-attends to all 8.

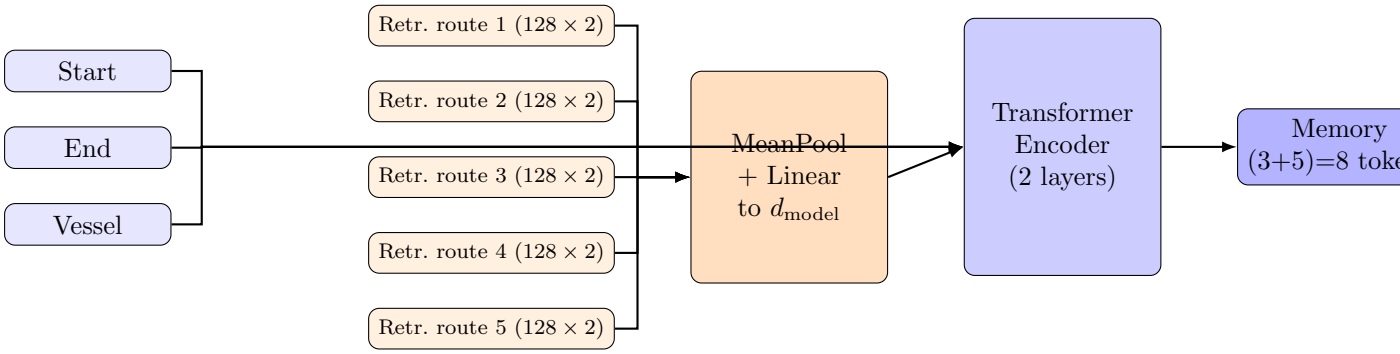


Figure 7: v9 architecture: each of the K=5 retrieved historical routes is compressed to a single d_{model}-dimensional vector by mean-pooling its 128 position vectors and projecting. The encoder memory grows from v4's 3 tokens to 8. The bottleneck is exactly the mean-pool step: per-step shape information is destroyed before the decoder ever sees it, which is why v9 loses to retrieval-top-1 on long routes (Table 5).

4.4.3 Results

Table 4: v9 results (epoch 38/60, `trajgen_128.npz`, $n = 500$, seed=42).

Method	ADE (m)	FDE (m)	norm. ADE
v9 ep38	**6207**	**0**	**0.0635**
Retrieval-top-1	6470	5603	0.0819
Great-circle	8226	0.1	0.0704
v5 ep60	6753	0	—

Table 5: Length-bucketed ADE for v9 (dirty data, $n = 500$, seed=42). v9 wins short and medium routes; retrieval-top-1 dominates long routes due to mean-pool collapse.

Bucket	n	v9 (m)	Retr-top-1 (m)	GC (m)
Short (<50 km)	243	**2117**	3366	2264
Medium (50–500 km)	250	**9050**	9143	11598
Long (>500 km)	7	46631	**18785**	94774

[1]This measurement was made on a slightly different evaluation slice from the full seed=42 harness; the corresponding seed=42 figure is 6470 m. Both are included in `results/summary.csv`.

v9 achieves the project's lowest overall ADE (6207 m). However, on the 7 long-route validation examples, retrieval-top-1 beats v9 by 2.5× (18785 vs. 46631 m). The **diagnosis**: mean-pooling each retrieved route to a single centroid-like d_{model} vector destroys per-step shape information; on long routes, the shape of historical traffic *is* the signal.

Figure 8 illustrates the diagnosis directly: on two long-bucket val routes, v9's prediction drifts hundreds of kilometres away from ground truth, while the verbatim retrieval-top-1 neighbour traces almost the right shape. The mean-pool encoder has summarised the neighbour into a single token whose content is invariant to its shape; from the decoder's perspective, all five retrieved tokens describe "a route from start to end" with no further detail.

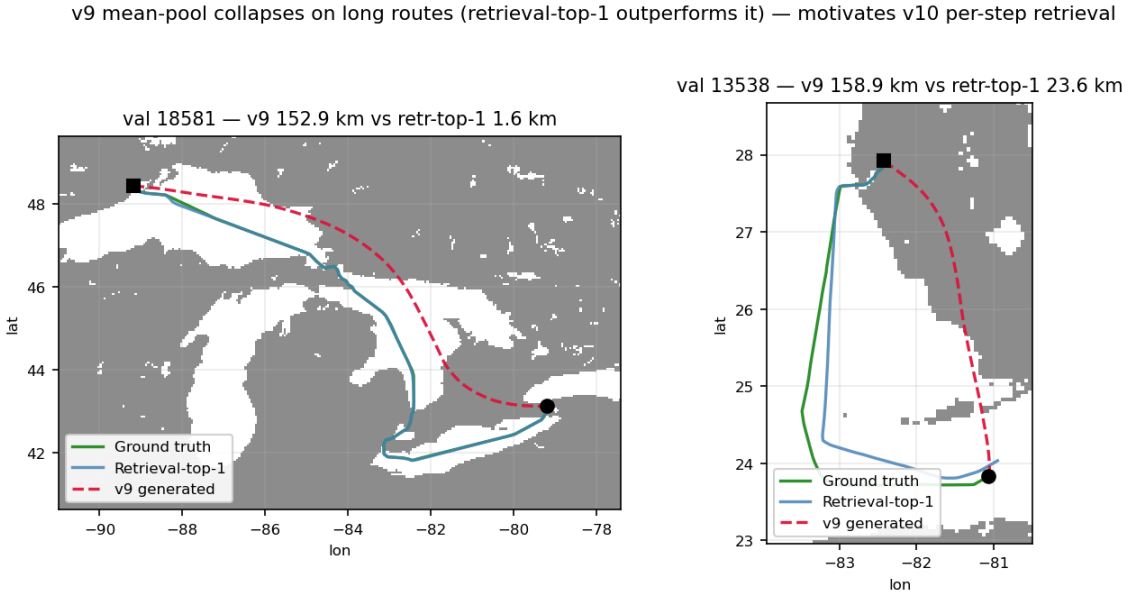


Figure 8: Two long-bucket val routes (>500 km) where v9 (red dashed) drifts far from ground truth (green), while the zero-training retrieval-top-1 neighbour (steelblue) tracks GT closely. Mean-pool collapse: v9 has the same five retrieved tokens that produced the steelblue route, but those tokens have been compressed to centroids before the decoder cross-attends to them. This motivates v10's per-step retrieval bank.

Training-duration note. `trajgen_v9_retrieval` ran for 42/60 epochs. The architectural-bottleneck conclusion (long-route ADE 7× short-route ADE; retrieval-top-1 winning the long bucket) was already firmly visible by epoch 38; the remaining epochs were dropped to free compute for v10 (whose architectural fix targets the same bottleneck). Epoch 38 is the checkpoint reported in Table 4.

4.5 v10 – Per-timestep retrieval and land-aware training

v10 makes three targeted changes to v9, implemented in `src/model_gen_v10.py`.

4.5.1 Change 1: Per-step retrieval bank

`MeanPoolRouteEncoder` is replaced with `PerStepRouteEncoder`. Each retrieved route is subsampled from $T = 128$ to $t_{\text{retr}} = 32$ points by uniform index linspace, projected per-point through a linear layer to d_{model}, and augmented with two learned embeddings: a *route-index embedding* (distinguishing retrieved routes 1–5) and a *step-index embedding* (distinguishing steps 0–31). Encoder memory grows from 8 tokens (v9) to $3 + 5 \times 32 = \mathbf{163}$ tokens. The decoder can now cross-attend to "neighbour k at step t" directly, preserving per-step route shape.

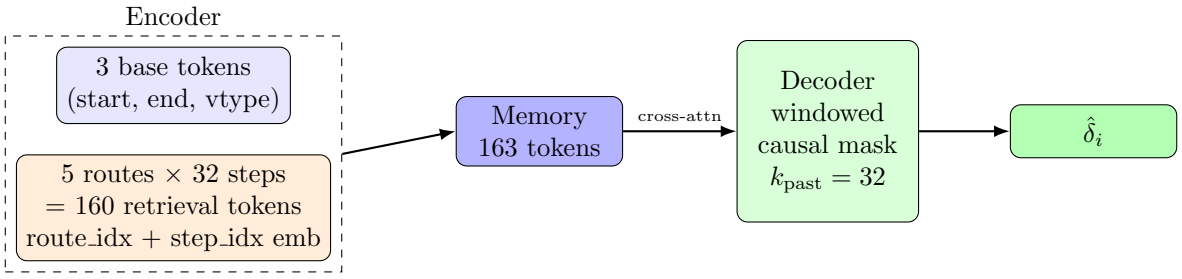


Figure 9: v10 encoder-decoder. The 163-token memory bank exposes per-step shape from each of the 5 retrieved routes, replacing v9's mean-pool collapse. The windowed causal mask (k_past = 32) limits self-attention drift over the 128-step rollout.

4.5.2 Change 2: Windowed causal mask

A windowed causal mask blocks attention to steps further than $k_{\text{past}} = 32$ in the decoder's own past:

$$M_{ij} = \begin{cases} 0 & \text{if } j \le i \text{ and } j \ge i - k_{\text{past}} \\ -\infty & \text{otherwise.} \end{cases}$$

This limits drift from stale self-state over 128 autoregressive steps.

4.5.3 Change 3: Land-aware loss and hard-projection inference

A differentiable land loss is added to Eq. (1):

$$\mathcal{L}_{\text{land}} = \frac{1}{T} \sum_i \text{ReLU}(\text{sdf\_km}(x_i^{\text{raw}}) - 10)^2 \,, \tag{3}$$

where sdf_km is sampled bilinearly from the precomputed 0.05° SDF raster (Figure 10). The threshold 10 km discounts rasterisation noise: the GT data in `trajgen_128.npz` itself has 10.83 % land fraction at 10 km, purely from GSHHS polygon clipping artefacts. The coefficient $\lambda_{\text{land}} = 0.01$ was chosen to keep the land gradient comparable to $\mathcal{L}_\delta$ without destabilising early training.

The full v10 loss is:

$$\mathcal{L}_{v10} = \mathcal{L}_\delta + 10\,\mathcal{L}_{\text{endpoint}} + 1\,\mathcal{L}_{\text{smooth}} + 0.01\,\mathcal{L}_{\text{land}}.$$

Differentiable SDF sampling. PyTorch's `F.grid_sample` backward is unimplemented on Apple Silicon MPS. A platform-independent bilinear sampler was therefore hand-rolled in `src/land_mask.py`: integer cell indices are detached (no gradient); floating-point offsets (f_r, f_c) carry the gradient through a manual bilinear combination:

$$v \approx (1-f_r)(1-f_c)\,v[r,c] + (1-f_r)f_c\,v[r,c{+}1] + f_r(1-f_c)\,v[r{+}1,c] + f_r f_c\,v[r{+}1,c{+}1].$$

Hard-projection at inference. After each decoder step, if sdf_km(pos) > 10, the position is projected to the nearest water cell via breadth-first search (BFS) in `LandMask.project_to_water` (20-cell search radius ≈ 100 km). The projected point is fed back into the decoder for the next step. After rollout, linear endpoint correction is applied; corrected points with sdf > 10 are re-projected.

4.5.4 Results on dirty data

Hard-projection drove land-crossing to near zero but incurred +42 % ADE. The WaterRouter snap-only post-processor (Section 4.6) reduced this penalty to +20 %; the root cause of both penalties—ground-truth data quality— is addressed in Section 4.7.

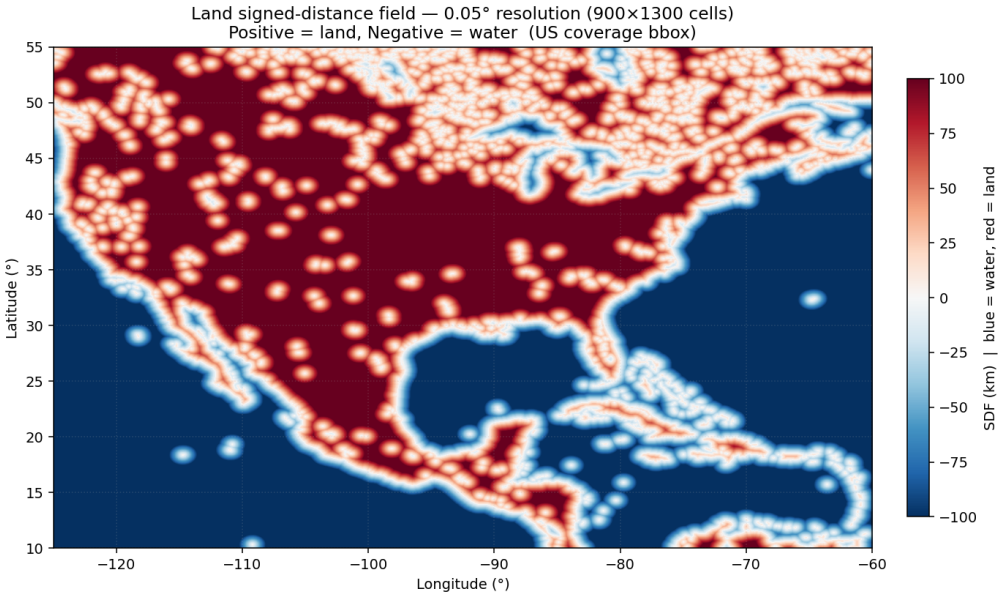


Figure 10: Land SDF raster at 0.05° ($\approx 5\,km$) resolution over the US coverage bbox. Blue = water (negative SDF), red = land (positive SDF). Saturated at $\pm 100\,km$ for visual clarity.

Table 6: v10 variants on dirty `trajgen_128.npz` ($n = 500$, seed=42).

Configuration	ADE (m)	land%@10km	cross%@0km
v10 raw	6584	7.15	36.2
v10 + hard-projection	9345	0.02	0.4
v10 + WaterRouter snap-only	7934	0.09	1.2
Retrieval-top-1	6470	11.03	35.6
Great-circle	8226	13.47	41.8

Figure 11 shows what "v10 raw still leaks land" looks like on two val examples, alongside the same prediction after WaterRouter snap-only. The raw outputs cut across small islands and short land bridges; the post-processor pushes each offending waypoint back to the nearest connected water cell. This is the qualitative justification for keeping a water-validity layer on top of the land-aware loss, even though the loss already discourages crossings on average.
Training-duration note (v10). `trajgen_v10` ran for 52/60 epochs. Training was halted once the val curves (Figure 14) plateaued and the land-crossing rate on the val set settled below 8 % — additional epochs would not change the "raw v10 leaks land" picture that the WaterRouter is built to repair. Epoch 52 is the production checkpoint quoted as "v10 ep52" throughout this report.

4.6 WaterRouter post-processor

The WaterRouter (`src/water_router.py`) is a non-learned post-processor that snaps v10 raw waypoints to water without retraining.

Fine-resolution graph. `scripts/data/build_water_graph.py` builds a 0.005° ($\approx 0.5\,km$) graph from the same GSHHS polygons. Each water cell carries a `neighbor_bits` mask encoding which of its 8 neighbours are also water. File size: ≈117 MB for the US bbox.

Snap-only mode (production). For each waypoint with sdf > 10: find the nearest fine-grid water cell satisfying (a) neighbor_bits > 0 (not a landlocked inlet artefact) and (b) coarse-SDF below threshold. Condition (a) prevents snapping to isolated one-pixel water artefacts

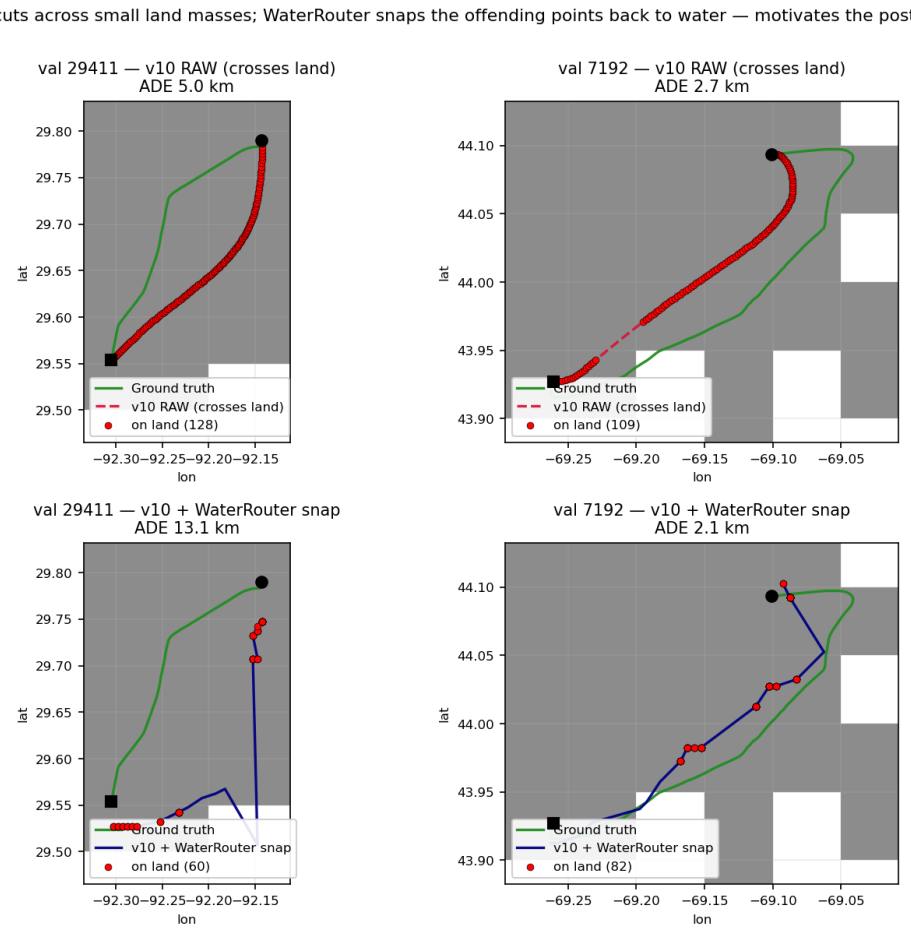


Figure 11: Two val routes where v10 raw output (top row, red dashed) crosses small land features that the differentiable land loss did not eliminate. Grey blocks are land masses (rasterised GSHHS coastline); white is water; green is the ground-truth route, red dashed is the v10 raw prediction, and navy solid (bottom row) is the same trajectory after WaterRouter snap-only — every offending waypoint moved to the nearest connected water cell on the 0.005° graph. The ADE penalty on dirty data is the price of also moving many *borderline* (coastal-river) waypoints; on clean data (Section 4.7, Table 7) the penalty essentially vanishes because few waypoints need snapping.

with no path to the open ocean.

Latency: $\approx 22\,ms$ per trajectory on a single CPU core. The bridging mode (`do_segment_bridge=True`), which uses A* search to re-route segments crossing land, was found to hurt ADE more than it fixed residual crossings and is disabled in production.

4.7 E1 – Upstream data cleaning

4.7.1 Diagnosis

Figure 12 shows two examples of what the "dirty" ground-truth data looks like: two AIS tracks from `trajgen_128.npz` whose pings sit mostly on inland water — rivers, estuaries, and harbour channels not represented in the GSHHS coarse coastline. These are not modelling artefacts; they are the supervisory targets that trained every v1–v9 model in this report.

Running `scripts/eval/land_crossing_diagnostic.py` on the *ground truth* trajectories in `trajgen_128.npz` returned: land%@10km = 10.83 %, cross%@10km = 36.0 %, cross%@0km = 87.6 %. The data itself was crossing land; the model trained on it was being asked to learn "do

Algorithm 2 WaterRouter Snap-Only

Require: trajectory **x** (128 waypoints), coarse SDF, fine-grid water graph

for $i = 1$ **to** 128 **do**
  if sdf_coarse(x_i) > 10 **then**
    $c \leftarrow$ fine-grid cell nearest to x_i
    BFS from c: find nearest c' with neighbor_bits(c') > 0 AND sdf_coarse(c') ≤ 10
    $x_i \leftarrow$ centre(c')
  end if
end for

return x

not cross land" while 36 % of its supervisory targets did exactly that.

4.7.2 The filter

`scripts/data/filter_trajgen_npz.py` retains only tracks satisfying:

$$\text{land5\_frac} \leq 0.02 \quad \text{AND} \quad \max_i \text{sdf\_km}(x_i) \leq 10,$$

where `land5_frac` is the fraction of waypoints with sdf > 5 km. The output, `trajgen_128_clean.npz`, contains 61,024 of the original 207,000 tracks (29.4 %).

4.7.3 Impact

Table 7: v10 ep52 weights re-evaluated on clean vs. dirty data.

Metric	Dirty data	Clean data	Change
GT land%@10km	10.83	**0.00**	−100 %
v10 raw ADE (m)	6584	**6278**	−4.6 %
v10 + hard-proj ADE	9345	**6579**	−30 %
v10 + WaterRouter ADE	7934	**6282**	−21 %
v10 + WaterRouter cross%@10km	1.2	**0.0**	−100 %
Retr-top-1 ADE	6470	12373	+91 %

The water-strict ADE penalty from snap-only essentially vanished on clean data (6282 m vs. 6278 m v10 raw on the same slice) because almost no waypoints needed snapping. The retrieval-top-1 regression on clean data is expected: the corpus shrank by 70 % and long-bucket sample count grew from 7 to 23, making KNN matches sparser.

The lesson: E1 establishes that the train/test distribution shift induced by the dirty corpus was responsible for a substantial fraction of v10's water-strict ADE cost. Data cleaning is therefore best understood as a *prerequisite* for fair architectural evaluation — it removes confounding training-set artefacts (38 % land-crossing in the ground truth itself) that would otherwise mask the contribution of v10's per-timestep retrieval and land-aware loss.

4.8 v11 – Halton-cone pointer (parked)

v11 replaced continuous displacement regression with a discrete pointer over a per-step candidate pool of water-only positions. Candidates were drawn by Halton quasi-random sampling [3] within a $[3\,km, 25\,km]$ forward cone aligned with bearing-to-destination. The model selected the best candidate via a masked softmax pointer head (`src/model_gen_v11.py`).

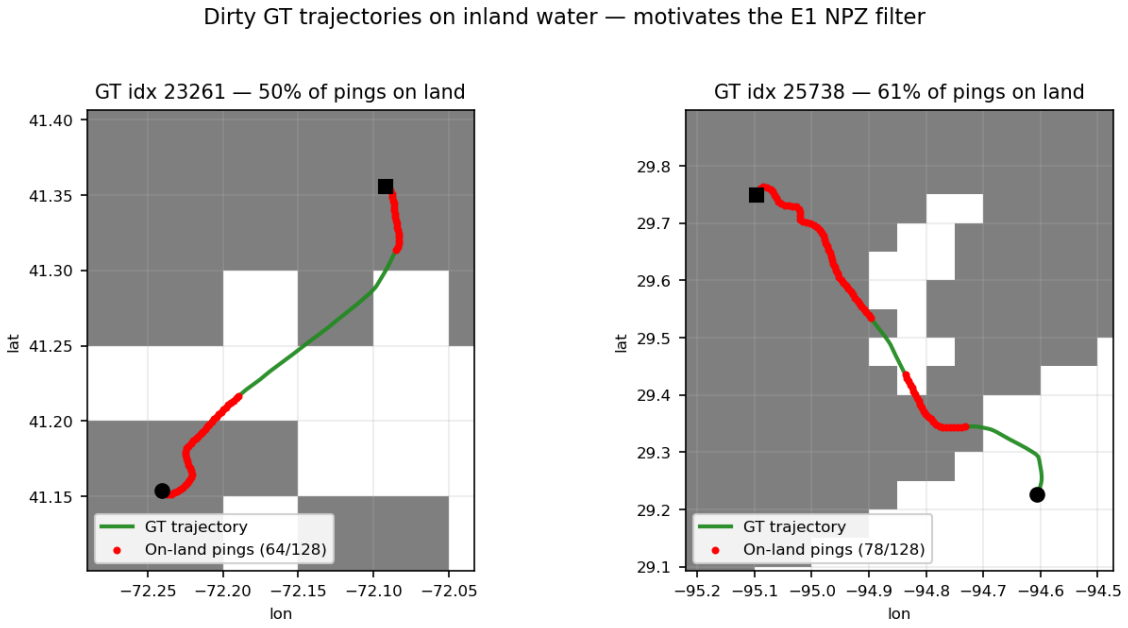


Figure 12: Two ground-truth trajectories from `trajgen_128.npz` that sit on inland water (rivers, harbour channels). Land is shaded grey, red dots flag GT pings whose coarse-SDF says "land". These trajectories are valid AIS recordings but live in waters that the 0.05° GSHHS-derived coastline does not represent, and they constitute roughly 10 % of all pings in the raw NPZ. Training any model that should "not cross land" on data like this is asking it to learn the opposite of its supervisory targets; the E1 filter (next subsection) removes them.

Result: ADE = 30000 m, path-length ratio 13×. **Diagnosis**: the median GT step after arc-length resampling to 128 points is 0.34 km; the cone minimum radius (3 km) was 9× larger. The model could only pick the smallest available candidate at every step—and even that overshot by almost an order of magnitude. **Lesson**: a candidate pool must be calibrated to the empirical step-size distribution of the training data, not to intuitions about "reasonable" step sizes. v11 is parked.

Training-duration note. `trajgen_v11_lite` ran its full 40/40 epochs. The failure is structural (step-size mismatch), not under-training; further epochs would not change ADE.

4.9 v12 – Water-cell K-ring pointer (abandoned)

v12 (`src/model_gen_v12.py`) addressed the v11 mismatch by deriving the candidate pool from the actual water-cell graph. At each decoder step, candidates are the Chebyshev-K ring of cells around the current cell plus a learned END token; land cells receive $-\infty$ logit before softmax, giving a *structural* water guarantee: the model cannot output a land cell. A small offset head regresses a continuous $(\Delta lon, \Delta lat)$ within the chosen cell for sub-km precision.

Table 8: v12 failure (40-epoch run, $K = 4$, $n = 500$, seed=42).

	v9 ep38	Retr-top-1	GC	v12 (ep40)
ADE (m)	6207	6741	8539	**27365**
FDE (m)	0	6160	0.1	41038
ADE long (m)	46631	22845	95900	236014
strict-0 cross%	—	—	—	81 %

Three structural failures:

1. **K = 4 silently drops long transitions.** P90/P95/P99 of GT cell transitions at 0.005° are 4/5/10 cells. Any transition with $|dx| > 4$ or $|dy| > 4$ maps to `ignore_index=-100` in cross-entropy, contributing zero gradient. The model never learns long jumps; at inference it is forced through short ring-steps, producing wandering routes (path-length ratio 1.48).

2. **CE on cell pointer is uncorrelated with rollout ADE.** Top-1 cell accuracy reached 87.2 % over 40 epochs while ADE remained catastrophic. $\lambda_{\text{ade}} = 0.001$ contributed $\sim 3.6 \times 10^{-5}$ to total loss vs. CE contributing ~ 0.545 ($50-100\times$ dominant). *Auxiliary losses must be calibrated by gradient magnitude, not intuition.*

3. **Argmax rollout drifts laterally.** Greedy argmax selection over 128 steps accumulates lateral offset that linear endpoint correction cannot undo.

v12 is abandoned; the post-mortem lives in `docs/decisions.md#v12`.

Training-duration note. `trajgen_v12` ran its full 40/40 epochs. Top-1 cell accuracy reached 87.2 % at the final epoch while ADE stayed 4.4× worse than v9 throughout — this is the per-step / rollout disconnect (Section 4, central negative finding) playing out a second time. The full schedule was run precisely to rule out "maybe more training would fix it"; it did not.

Figure 13 shows what v12's output looks like in practice: predicted trajectories trace short, axis-aligned cell-to-cell moves and accumulate lateral drift over the 128-step rollout. The K=4 ring (the second-failure column in the post-mortem above) prevents the larger diagonal jumps that real GT routes make; greedy argmax then chooses whichever ring cell is roughly toward the destination, producing the Manhattan zigzag visible in every panel.

5 Results and Discussion

5.1 Summary of all trajectory-generation experiments

Figure 5 provides a visual overview of ADE across all trajectory-generation model versions. With linear endpoint correction, all trained ADE-target models reach FDE ≈ 0 by construction; the FDE discriminator is meaningful only for retrieval-top-1 (FDE = 5603 m, no correction applied) and for v10 + WaterRouter (small non-zero offset from snapping at the final waypoint).

5.2 Cross-cutting findings

1. **Per-step validation loss is uncorrelated with rollout ADE.** The architectural ladder v2→v5 improved val_delta by 14 % (0.086 → 0.074) with zero real-world ADE improvement. The same pattern appeared in v12: CE top-1 accuracy reached 87.2 % while ADE was 4.4× v9. Only end-to-end rollout evaluation is informative.

2. **Retrieval is the only architectural innovation that moved the needle.** The retrieval-top-1 baseline (zero parameters) beat the best trained AR model (v5) by 19 % on ADE. v9's trained model improved on retrieval-top-1 by 4 % overall. All other architectural elaborations (bigger model, bearing features, scheduled sampling, land loss, windowed mask) produced no significant overall ADE gain.

3. **Data quality dominates model quality.** Filtering the training set (E1, Section 4.7) reduced the water-strict ADE by 21 % — larger than any model change. The land-crossing penalty from hard-projection collapsed from +42 % to +5 % after cleaning.

4. **Constraint losses need constraint-violating examples.** Both the obstacle loss (v6/v7) and the land loss (v10) require training examples where the constraint *would* be violated to generate a non-zero gradient. The obstacle loss never fired structurally; the land loss fired inconsistently on dirty data (threshold too close to GT's own land rate).

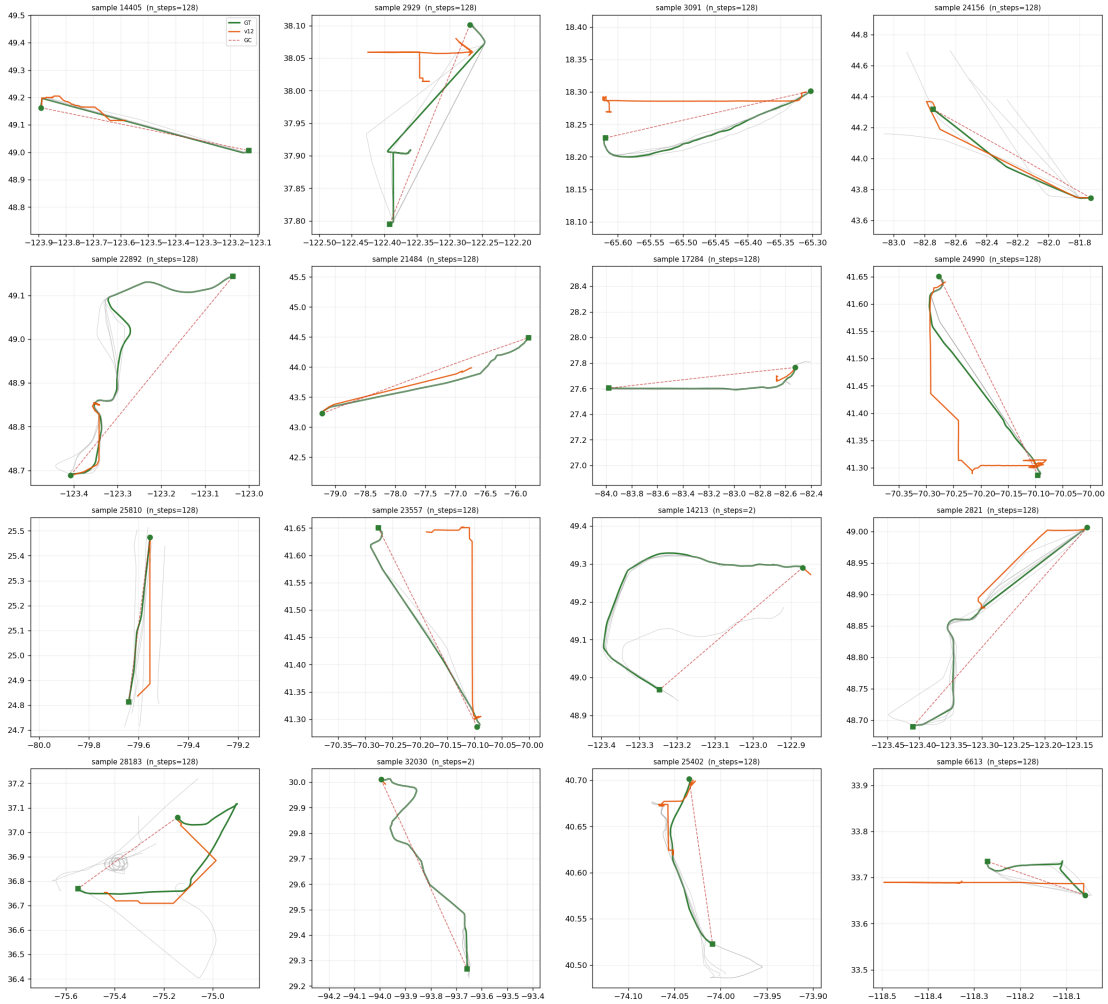


Figure 13: v12 cell-pointer output on a grid of val routes (this is the same `viz_v12.png` qualitative panel from `scripts/eval/visualize_gen_v12.py`). Green = GT, blue = retrieved neighbours, red dashed = v12 prediction. The characteristic axis-aligned zigzag is structural: the K-ring candidate set is mostly cardinal-direction neighbours, and 128 sequential argmax picks compound into wandering routes with path-length ratio 1.48.

5. **Structural guarantees are decoupled from trajectory quality.** v12's masked-logit pointer guaranteed cross%@0 = 81 % — better than its own ground truth (87 %) — while producing ADE 4.4× worse than v9. Water-validity and route accuracy are distinct objectives requiring separate engineering.

5.3 Production system use-case ranking

The production system has no single model that simultaneously achieves lowest overall ADE, lowest long-bucket ADE, and strict water-validity. This three-way trade-off is the primary motivation for the v13 design in Section 7.

6 Summary and Conclusion

6.1 What was achieved

Starting from a next-step displacement predictor that could not beat a constant-velocity baseline, this project arrived at:

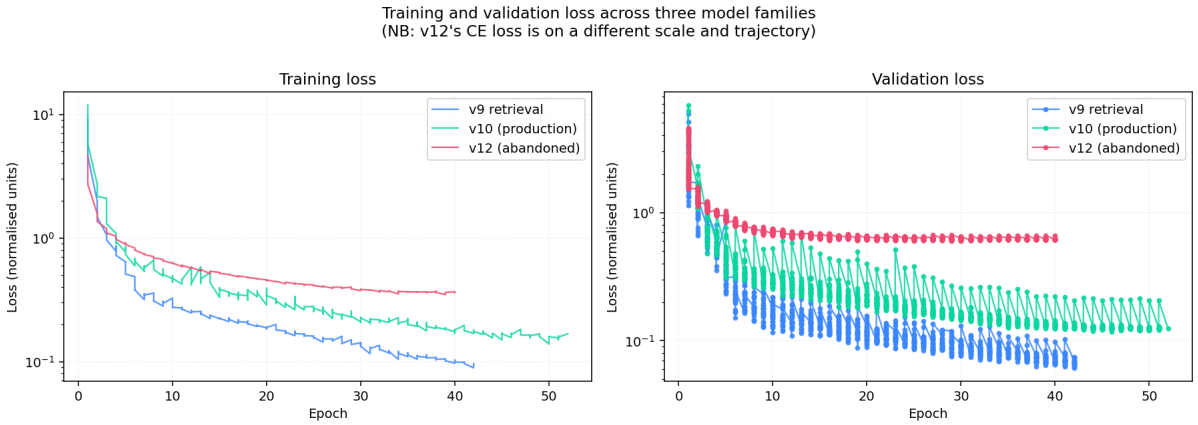


Figure 14: Training and validation loss for v9 (retrieval), v10 (production), and v12 (abandoned) on log scale. Note that v12's CE loss is on a fundamentally different scale and not comparable to v9/v10's regression loss.

Table 9: Use-case ranking for the production system.

Use case	Configuration	ADE (m)	land%@10km
Water-strict deployment	v10+router, clean data	**6282**	**0.00**
Best ADE, no constraint	v9 ep38, dirty data	**6207**	12.81
Best long-route (>500 km)	Retr-top-1, dirty	18785*	11.03
Geometric baseline	Great-circle	8226	13.47

*Long bucket only ($n = 7$); overall retr-top-1 ADE is 6470 m.

- A systematic study of **next-step displacement prediction** (v1) covering causal vs. bidirectional attention, displacement vs. velocity targets, vessel-type embeddings, and lighthouse / nearest-vessel auxiliary features — yielding the negative result that constant velocity is a near-optimal baseline on post-turn-segmented open-sea tracks, and the methodological lesson that motivated the move to full-sequence generation.
- An **architectural ladder for conditional route generation** (v2–v5) including a two-pass parallel formulation of scheduled sampling, bearing-feature conditioning, and a linear endpoint correction that drives FDE to zero without retraining.
- **Retrieval-augmented decoding** (v9, v10) — per-timestep retrieval over a 163-token memory combined with a windowed causal mask and a differentiable land loss computed by bilinear interpolation on a 0.05° signed-distance raster.
- A production route generator (v10 + WaterRouter) achieving ADE = 6282 m with 0 % land-crossing at the 10 km threshold, running at ∼50 ms per query on a CPU.
- Reusable infrastructure: a differentiable land SDF, a fine-resolution water-cell graph and A*/BFS router, and a retrieval substrate that can be reused by any future model (including v13).
- A full data pipeline from raw NOAA AIS downloads to evaluation-ready NPZ datasets, including E1 land filtering and KNN-cache construction.
- Detailed post-mortems for six failed or parked model variants (v6/v7/v8, v11, v12), surfacing reusable design principles for future generative-routing work.

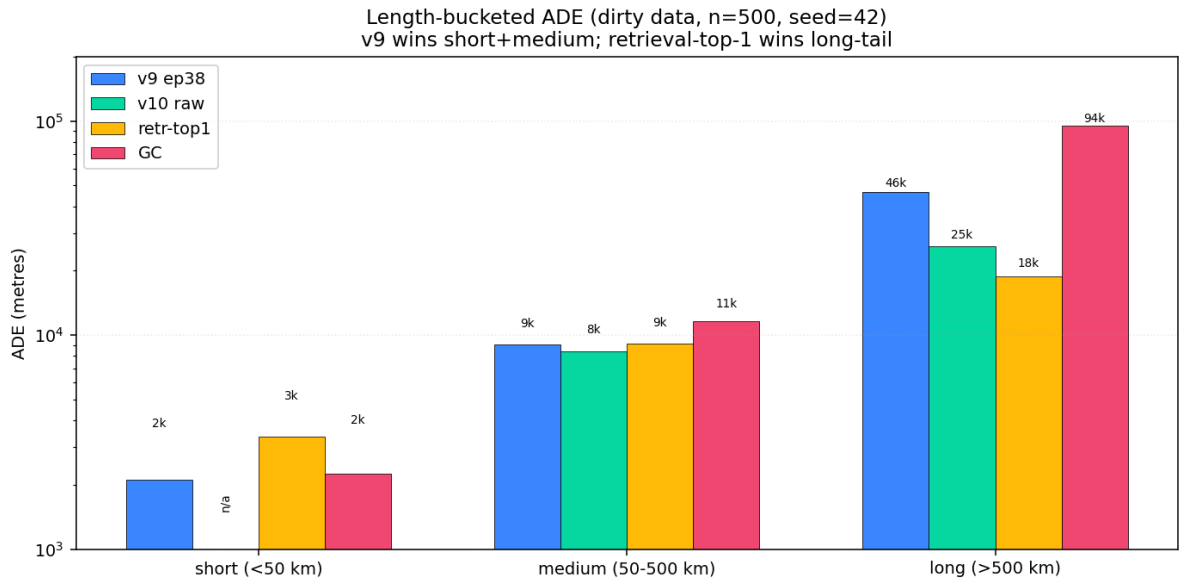


Figure 15: Length-bucketed ADE (log scale). v9 wins short and medium; retrieval-top-1 dominates the long-tail (>500 km). No single model wins all three buckets — the key motivation for v13's length router.

6.2 Open challenges

Three structural problems remain unsolved:

1. **Multimodality.** ADE measures distance to a single ground-truth route. A model that generates one high-quality mode of the route distribution will appear worse than one that generates the mean, even if the mode is more useful. Addressing this requires a coverage metric and a generative (non-deterministic) model — motivating v13.

2. **Long-route degradation.** No trained model has matched retrieval-top-1 on routes >500 km. The mean-pool failure (v9) was fixed in v10, but v10's long-bucket ADE remains 1.4× worse than retrieval-top-1 on dirty data. A length router partially mitigates this without training.

3. **Hard water-validity at zero ADE cost.** All current approaches pay an ADE cost for enforcing the water constraint: +5 % for hard-projection on clean data, ≈0 % for snap-only on clean data (production). A model that structurally cannot generate land-crossing routes while maintaining ADE competitive with v9 remains an open problem.

6.3 The project's deepest lesson

The single most repeated finding is: **per-step training metrics do not predict autoregressive rollout quality.** v2→v5 demonstrated this on val_delta; v12 demonstrated it again on CE accuracy. Any proposed architecture should be validated by rollout evaluation at the smallest reasonable scale before committing to a full training run. This principle directly informs the v13 pre-flight checklist (Section 7).

7 Outlook: Proposed v13 – TrajDiff

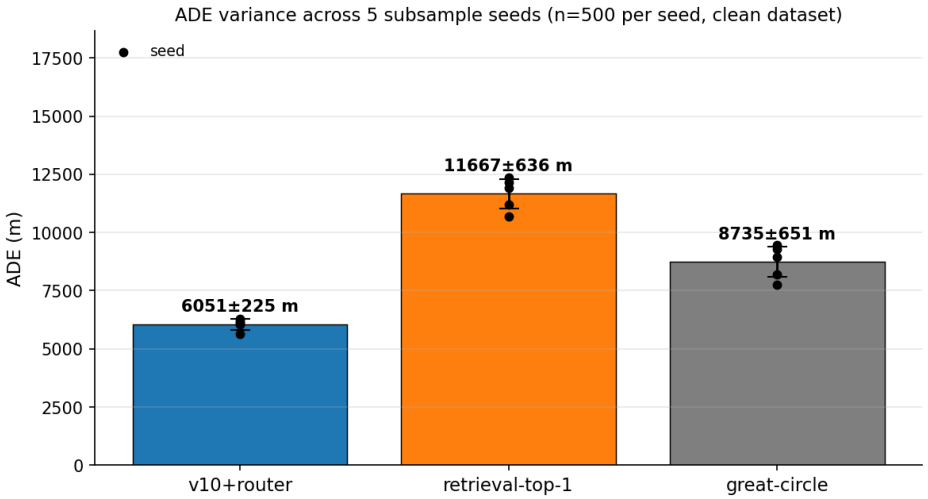


Figure 16: Production ADE variance across five subsample seeds ($\{0, 1, 2, 42, 123\}$, $n = 500$ each) on clean data. Black dots are individual seeds; error bars are $\pm 1\sigma$. The headline seed=42 figure (6282 m) lies inside the v10 + WaterRouter $\pm\sigma$ band (6052 ± 225 m), so the production claim is not seed-cherry-picked. Retrieval-top-1 degrades sharply on clean data because the clean corpus is 70 % smaller than the dirty one — nearest-neighbour retrieval depends on corpus density. Full per-seed numbers are in Appendix C.

7.1 Motivation

The v10 + WaterRouter production system is autoregressive: it generates one waypoint at a time, conditioning on all previously generated waypoints. This structure couples error propagation across all 128 steps (the source of long-route degradation) and cannot natively sample multiple diverse routes (limiting coverage of the multimodal distribution).

TrajDiff abandons autoregressive decoding in favour of *whole-sequence denoising diffusion*, reframing route generation as: "start from a Gaussian noise trajectory and iteratively denoise it into a plausible route, conditioning on (start, end, vessel-type, 5 retrieved routes)." Diffusion models naturally support: (1) multiple independent samples per query (diversity without retraining), (2) inference-time guidance (land penalty applied at each DDIM step), (3) the same conditioning bank as v10 (reusing verified retrieval infrastructure).

7.2 Architecture overview

A DiT [8] backbone with 6 bidirectional self-attention blocks ($d = 256$, 8 heads, 1024-dim FFN, adaLN-Zero modulation by diffusion timestep). The conditioning memory (same 163-token v10 bank) is provided via cross-attention. SDF features (5×5 coarse patch + 3×3 fine patch at each of 128 noisy positions) are computed per block and concatenated with the per-step input. Classifier-free guidance with $p_{\text{drop}} = 0.1$ and $w = 1.5$. Score-based land guidance adds $\eta_t \cdot \nabla_{x_t} \text{ReLU}(\text{sdf} - 5)^2$ at each DDIM step, with η_t annealed from 0 to 0.1. At inference, 8 trajectories are sampled in parallel and the best-scoring is returned.

7.3 Length router (v13a)

Independent of the trained model, a no-training length router dispatches on great-circle distance:

$$\hat{\mathbf{x}} = \begin{cases} \text{retrieval-top-1} & \text{if } d_{\text{GC}} > 500\,km \\ \text{v10 raw} & \text{otherwise} \end{cases}$$

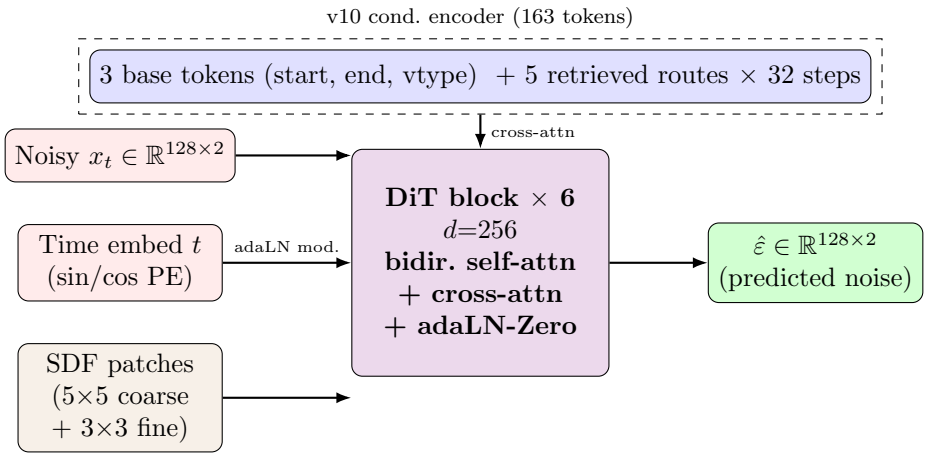


Figure 17: v13 TrajDiff architecture (planned; not yet implemented). A bidirectional DiT denoises a 128-point noisy trajectory in normalised lon/lat space, conditioning on the same 163-token retrieval bank used by v10 (cross-attention) and on multi-scale SDF features sampled at the noisy positions (concatenated with the time embedding). At sample time, 8 trajectories are drawn in parallel, ranked by distance to the retrieval-top-1 neighbour minus a penalty for land crossings, and the best returned. The conditioning encoder (top) is reused verbatim from v10 (Figure 9), so the v10 retrieval substrate transfers into the diffusion setting without retraining the encoder.

Projected ADE: 5500–5800 m on dirty data, based on the per-bucket performance of v10 raw and retrieval-top-1 reported in Section 5. This baseline must be evaluated before committing to v13 training: if the length router already clears the target, the full diffusion model is unnecessary.

7.4 Pre-flight checklist

The engineering specification in `docs/v13_proposal.md` lists five pre-flight checks required before the 100-epoch training run:

1. Re-evaluate v10 across seeds $\{0, 1, 7, 42, 100\}$ on clean data to measure σ_{ADE} (required to determine statistical significance of any v13 improvement).
2. Implement and evaluate the length router (v13a) without the trained model.
3. Build `land_sdf_005deg.npz` and verify memory budget (≈2.3 GB at float16).
4. Train a minimal unconditional DiT on $T = 16$ (downsampled trajectories) for 20 epochs to confirm diffusion dynamics before scaling.
5. Only then scale to $T = 128$ with full conditioning.

v13 is specified but not yet implemented. All code described in this section (`src/model_gen_v13.py`, `src/diffusion.py`, training and eval scripts) remains to be written.

References

[1] Samy Bengio, Oriol Vinyals, Navdeep Jaitly, and Noam Shazeer. Scheduled sampling for sequence prediction with recurrent neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 28, 2015.

[2] Georgios Grigoropoulos, Giannis Spiliopoulos, Ilias Chamatidis, Manolis Kaliorakis, Alexandros Troupiotis-Kapeliaris, Marios Vodas, Evangelia Filippou, Eva Chondrodima, Nikos Pelekis, Yannis Theodoridis, Dimitris Zissis, and Konstantina Bereta. A scalable system for maritime route and event forecasting. In *Proceedings of the 27th International Conference on Extending Database Technology (EDBT)*, page 8 pages, Paestum, Italy, March 2024. ISBN 978-3-89318-095-0.

[3] John H. Halton. On the efficiency of certain quasi-random sequences of points in evaluating multi-dimensional integrals. *Numerische Mathematik*, 2:84–90, 1960.

[4] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 6840–6851, 2020.

[5] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. In *NeurIPS Workshop on Deep Generative Models and Downstream Applications*, 2021.

[6] Huanhuan Li, Hang Jiao, and Zaili Yang. Ship trajectory prediction based on machine learning and deep learning: A systematic review and methods analysis. *Engineering Applications of Artificial Intelligence*, 126:107062, 2023.

[7] National Oceanic and Atmospheric Administration (NOAA). Automatic Identification System (AIS) – US Vessel Traffic Data. MarineCadastre.gov, 2024. Accessed 2024. Dataset covers US coastal waters.

[8] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.

[9] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations (ICLR)*, 2021.

[10] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017.

[11] Paul Wessel and Walter H. F. Smith. A global, self-consistent, hierarchical, high-resolution shoreline database. *Journal of Geophysical Research: Solid Earth*, 101(B4):8741–8743, 1996.

A Reproducibility Notes

The production result (v10 + WaterRouter on clean data, ADE 6282 m, 0 % land@10km) can be reproduced from the repository root as follows.

Step 1: One-shot data builds (cached; skip if files exist):

```
python3 scripts/data/filter_trajgen_npz.py \
    --out_npz data/processed/trajgen_128_clean.npz \
    --land5_max_frac 0.02 --max_pen_km 10.0

python3 -c "from␣src.data_gen_retrieval␣import␣build_and_cache_knn;␣\
␣␣␣␣build_and_cache_knn('data/processed/trajgen_128_clean.npz',␣\
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣'data/processed/trajgen_128_clean_knn_k5.npz',␣\
␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣␣val_frac=0.15,␣seed=42,␣k=5,␣vtype_weight=0.5)"
```

Step 2: Production evaluation (v10 ep52 checkpoint, no retraining):

```
python3 scripts/eval/eval_gen_v10_router.py \
    --data_npz data/processed/trajgen_128_clean.npz \
    --checkpoint runs/trajgen_v10/best.pt \
    --knn_cache data/processed/trajgen_128_clean_knn_k5.npz \
    --water_graph data/processed/water_graph_005deg.npz \
    --land_sdf data/processed/land_sdf_050deg.npz \
    --n_eval 500 --no_frechet
```

Step 3: Regenerate report figures:

```
python3 scripts/eval/plot_comparison.py --out_dir figures/
python3 scripts/eval/plot_land_sdf.py \
    --land_sdf data/processed/land_sdf_050deg.npz \
    --out figures/land_sdf.png
cp figures/*.png report/figures/
cd report && latexmk -pdf report.tex
```

Multi-seed variance. The single-seed limitation flagged in earlier drafts has been resolved: `scripts/eval/multi_seed_variance.py` now sweeps seeds $\{0, 1, 2, 42, 123\}$ on clean data and produces `results/seed_variance.csv`, `results/seed_variance_summary.txt`, and `figures/seed_variance.png` (Appendix C).

Remaining limitations:

- Variance is over subsample selection ($n = 500$ draws from the full val pool of $\sim$8.8 k routes), not over training seeds. Only one v10 training was carried out.
- The displacement model (v1) baseline uses `results/baseline.json` saved per run; trajectory generation models do not produce this file (different evaluation protocol).
- The `12mo_seq80.yaml` displacement config has no corresponding run (`runs/12mo_seq80/` is empty) — this run was never executed.

B Repository Structure

Key paths relative to the repository root:

`src/` Model and data modules. `model.py`/`data.py`/`metrics.py` (v1); `*_gen.py` (v2); `*_gen_obs.py` (v6/v7); `*_gen_retrieval.py` (v9); `model_gen_v10.py`/`land_mask.py`/`water_router.py` (v10, production); `model_gen_v11.py`/`candidates.py` (v11, parked); `model_gen_v12.py`/`data_gen_v12.p` (v12, abandoned).

`scripts/train/` One training script per model version.

`scripts/eval/` Evaluation, visualisation, and plotting scripts.

`configs/` YAML experiment configs grouped by approach (`displacement/`, `trajgen/`, `obstacle/`, `retrieval/`).

`runs/` Training outputs (not tracked by git): `best.pt`, `metrics.csv`, `training_curves.png` per run.

`data/processed/` Processed datasets and cached artefacts (not tracked).

`docs/` `narrative.md` (project narrative §1–§8); `decisions.md` (post-mortems); `v13_proposal.md` (engineering spec); `references/` (three reference PDFs and notes).

`results/summary.csv` One row per completed experiment with ADE/FDE and qualitative notes.

C Multi-seed variance of the production result

The headline production result reported in Section 5.3 (ADE = 6282 m for v10 + WaterRouter snap-only on `trajgen_128_clean.npz`, n=500, seed = 42) is supported by four additional seed runs. Each seed selects a different 500-route subsample from the same val pool, so the figures probe how sensitive the headline number is to which 500 routes happened to be drawn.

Concretely, `scripts/eval/multi_seed_variance.py` takes the union of five 500-route subsamples (seeds $\{0, 1, 2, 42, 123\}$, totalling 2240 unique val routes out of 8783), runs v10 inference and the WaterRouter snap once on the union, then slices to each seed and recomputes ADE, FDE, and the 10 km land-crossing rate. Table 10 reports the per-seed numbers; Figure 16 (in the main text) plots the production ADE with $\pm 1\sigma$ error bars.

Table 10: Per-seed evaluation of the three core methods on clean data. v10 + WaterRouter and retrieval-top-1 both reach exactly 0 % land-crossing (the corpus and the post-processor are water-structural). Means are unweighted over the five seeds.

seed	v10 + WaterRouter			Retrieval-top-1			Great-circle		
	ADE	FDE	cross%	ADE	FDE	cross%	ADE	FDE	cross%
0	6035	575	0.0	12170	12428	0.0	9459	0.1	9.4
1	6106	594	0.0	10680	11408	0.0	8202	0.1	8.0
2	5633	567	0.0	11190	11234	0.0	7763	0.1	8.0
42	6282	561	0.0	12374	12283	0.0	9304	0.1	9.0
123	6201	606	0.0	11922	11809	0.0	8949	0.1	9.6
mean ± std	6052 ± 225	581 ± 17	0.0	11667 ± 636	11832 ± 469	0.0	8735 ± 651	0.1	8.8 ± 0.7

What this rules out. Any future improvement of less than ∼225 m in ADE is not statistically distinguishable from this seed noise floor; report claims below that bar should not be reported as headline improvements.

What this still does not measure. This is variance over *which 500 routes were drawn*, not over training seeds (only one training seed was used for v10). Measuring training-seed variance would require multiple full v10 trainings, which exceeds the project's compute budget; it is recorded as future work in the v13 pre-flight checklist (Section 7).